# TyProX Engine

## Full Technical Architecture, UI/UX Design & System Blueprint (A-Z)

**Document Version:** 1.0.0 (Production Blueprint)
**Target Platform:** Next.js 16.2 (App Router) / React 19 / Supabase / TailwindCSS v4
**Author:** TyProX Core Engineering Team
**Scope:** Complete end-to-end breakdown from UI design system, Web Worker state engine, anti-cheat math, real-time database schema, to API endpoints and deployment.

**Executive Summary:**
TyProX is a state-of-the-art, high-frequency analytical typing platform and gamified ecosystem designed for hyper-speed text input, micro-second telemetry processing, and deep performance diagnostics. Unlike traditional web applications that calculate statistics in the main rendering thread, TyProX decouples user input capture from analytical evaluation using a dedicated Web Worker (analytics.worker.ts). Coupled with Supabase PostgreSQL Row-Level Security (RLS), real-time WebSocket subscriptions, dynamic collectible companion pets, and anti-cheat timing analysis, TyProX offers an uncompromised competitive environment.

## Table of Contents

# 1. Executive Overview & Architectural Philosophy

TyProX was built to solve the modern performance issues found in web-based typing software. Traditional platforms suffer from input frame drops during long runs due to synchronous state recalculations on every single keypress. TyProX introduces three fundamental engineering tenets:

- **Zero-Input Latency:** Keystroke events are captured in an un-lagged focus trap textarea, immediately echoed locally, and dispatched to background Web Workers.
- **Rich Modern Aesthetics:** Built using a cohesive Dark/Light glassmorphism design system with custom HSL tokens, dynamic radial ambient glows, and fluid Framer Motion transitions.
- **Comprehensive Gamification & Analytics:** Features collectibles (Virtual Cat Companions), multi-tier leaderboards (Grandmaster to Bronze), full replay playback, and detailed key/bigram bottleneck speed analysis.

# 2. System Architecture & Core Technology Stack

| Layer | Technology | Role & Description |
|---|---|---|
| Framework | Next.js 16.2.9 (App Router) | Server Component rendering, API routes, SEO route generation |
| UI Library | React 19.2.4 | Concurrent mode UI components, hooks, ref management |
| Language | TypeScript 5.x | End-to-end type safety, Zod validation models, strict mode |
| Styling Engine | TailwindCSS v4 + CSS Variables | Design token system, glassmorphism utilities, dark/light themes |
| State Engine | Zustand v5 | Client-side reactive store with local persistence middleware |
| Background Worker | Web Worker API (Worker) | High-frequency telemetry math, WPM/RSD calculation, timeline builder |
| Database & Auth | Supabase PostgreSQL + SSR | Row-Level Security (RLS), OAuth (Google/GitHub), Realtime subscriptions |
| Visualizations | Recharts 3.8.1 + Framer Motion | Second-by-second speed timeline graphs and layout animations |

# 3. UI/UX Design Engine & Component Architecture

The TyProX visual layer emphasizes high accessibility, low visual fatigue, and instant user feedback. Key components include:

### • TypingContainer (components/typing/typing-container.tsx)

Main engine shell containing top configuration bar (mode/duration selectors), real-time glass statistics cards, transparent input focus-trap textarea, and ambient dynamic radial background glow scaling in size and opacity with user WPM.

### • TextDisplay (components/typing/text-display.tsx)

Viewport rendering target sentences broken down into tokenized words and individual character spans. Features smooth auto-scrolling to active lines, blinking vertical accent caret, and distinct styling states for untyped, correct, and wrong characters.

- **ResultScreen (components/typing/result-screen.tsx)**

Post-test performance dashboard featuring animated metric counters, speed/accuracy timelines powered by Recharts, error key diagnostics, canvas confetti triggers, and shareable challenge lobby creation.

- **ReplayPlayer (components/typing/replay-player.tsx)**

Interactive keystroke reproduction tool utilizing requestAnimationFrame to replay exact timing telemetry with progress scrubbing and 1x to 4x playback speed toggles.

- **CatCompanion (components/typing/cat-companion.tsx)**

Dynamic virtual cat assistant featuring 12 unique collectible personalities, custom Framer Motion animations (tail wags, ear twitches, aura pulses), and contextual reactive dialogues based on typing metrics.

- **LeaderboardPage (app/leaderboard/page.tsx)**

Global ranking leaderboard displaying user rank tiers (Grandmaster to Bronze), timeframe filters (all-time, weekly, daily), and real-time Supabase Postgres change listening.

# 4. State Management & Decoupled Web Worker Telemetry

State management in TyProX is cleanly split across three specialized Zustand stores and a dedicated background Web Worker script:

## Store Architecture

1. **useTypingStore (stores/typing-store.ts):** Manages test configuration (duration, mode, seed), current live status ('idle' | 'running' | 'completed' | 'paused'), real-time stats (WPM, accuracy, combo), and dispatches raw keystrokes to the worker.
2. **useUserStore (stores/user-store.ts):** Manages auth sessions, user profile data, local guest result history (up to 100 runs), and preference toggles (theme, mono fonts). Persisted via Zustand local storage middleware.
3. **useToastStore (stores/toast-store.ts):** Lightweight notification system for system alerts and unlock popups.

## Web Worker Engine (workers/analytics.worker.ts)

During typing, every keypress dispatches an asynchronous message to the worker. The worker maintains an O(1) running position map (`Map`) of character correctness. On every keystroke, it calculates instantaneous WPM and accuracy, returning a lightweight `TICK` message. Upon test completion, it executes a heavy `FINALIZE` pass:

**FINALIZE Execution Pass:**
• **Keystroke Filter & Monotonicity Check:** Verifies timing array integrity.
• **Relative Standard Deviation (RSD) Consistency:** Computes interval standard deviation across correct keypresses to rate typing consistency (0-100%).
• **Second-by-Second Timeline Builder:** Generates precise point-in-time WPM graphs for rendering in Recharts.
• **Performance Diagnostics:** Aggregates per-key speed averages, error key frequencies, and two-letter bigram transition speeds (e.g., 'th', 'in', 'er').

# 5. Database Architecture & Full PostgreSQL Schema

TyProX utilizes Supabase PostgreSQL with strict Row Level Security (RLS), triggers for user profile generation, and custom indexed views for instant leaderboard aggregation.

| | |
|---|---|
| **public.profiles** | Stores user profile data (id UUID REFERENCES auth.users, username VARCHAR(30) UNIQUE, display_name, avatar_url, theme, font_family). Auto-created on auth sign-up via trigger. |
| **public.test_results** | Stores main test scores (id UUID, user_id UUID, wpm REAL, raw_wpm REAL, accuracy REAL, consistency REAL, error_count INT, backspace_count INT, mode, duration, seed, is_invalidated). |
| **public.replays** | Stores compressed JSONB keystroke telemetry payload linked 1:1 to test_results via test_result_id CASCADE. |
| **public.streaks** | Tracks daily typing streaks per user (current_streak INT, longest_streak INT, last_active_date DATE). |
| **public.daily_challenges** | Contains daily scheduled challenges (challenge_date DATE UNIQUE, mode, duration, seed, text_content). |

| | |
|---|---|
| **public.challenge_links** | Custom multiplayer challenge lobbies (creator_id, creator_wpm, creator_accuracy, mode, duration, seed, expires_at). |
| **public.achievements** | System achievements dictionary (code VARCHAR UNIQUE, name, description, criteria JSONB, icon_path). |
| **public.user_achievements** | Pivot table mapping unlocked achievements to users (user_id, achievement_id, unlocked_at). |

## Leaderboard SQL View & Auto-Profile Trigger

```
-- PostgreSQL View: Highest WPM run per user, mode, and duration
CREATE OR REPLACE VIEW public.leaderboard_view AS
SELECT DISTINCT ON (user_id, mode, duration)
id, user_id, wpm, accuracy, consistency, mode, duration, is_invalidated, created_at
FROM public.test_results
WHERE is_invalidated = false
ORDER BY user_id, mode, duration, wpm DESC;

-- Auto-profile & streak generator trigger on new user sign-up
CREATE OR REPLACE FUNCTION public.handle_new_user() RETURNS TRIGGER AS $$
BEGIN
INSERT INTO public.profiles (id, username, display_name, avatar_url)
VALUES (
NEW.id,
COALESCE(NEW.raw_user_meta_data->>'username', 'user_' || substring(NEW.id::text, 1, 8)),
COALESCE(NEW.raw_user_meta_data->>'display_name', NEW.raw_user_meta_data->>'full_name'),
NEW.raw_user_meta_data->>'avatar_url'
);
INSERT INTO public.streaks (user_id) VALUES (NEW.id);
RETURN NEW;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

# 6. API Specifications & Server Security Matrix

TyProX enforces strict server-side validation using Next.js API route handlers and Zod schema parsing:

## POST /api/results Route Breakdown

- **Schema Validation:** Incoming payload validated against Zod schema (WPM capped at 350, telemetry array capped at 10,000 events).
- **Anti-Cheat Engine:** Analyzes millisecond timestamps. Rejects scores if timeline anomalies are detected or if timing standard deviation is below 1.5ms (mechanical autotyper detection).
- **Streak Maintenance:** Calculates day difference between current date and `last_active_date`. Increments streak if exactly 1 day apart, resets to 1 if >1 day.
- **Achievement Evaluation:** Dynamically queries `achievements` table and unlocks criteria matching speed thresholds or streak milestones.

## Security & Row Level Security (RLS) Policy Matrix

| Table | Operation | RLS Policy Rule |
|---|---|---|
| profiles | SELECT / UPDATE | Readable by everyone; update restricted to auth.uid() == id |
| test_results | INSERT | With check (auth.uid() == user_id OR user_id IS NULL) |
| replays | INSERT | Requires existing parent test_result owned by caller |
| streaks | UPDATE | Restricted to auth.uid() == user_id |
| challenge_links | INSERT | Authenticated users can create challenge link lobbies |

# 7. Anti-Cheat Mechanics & Mathematical Formulas

TyProX uses rigorous standardized mathematical formulas to compute typing performance metrics:

- **Net Words Per Minute (WPM):**

```
WPM = ((Correct Characters) / 5) / (Time in Minutes)
```

- **Raw Words Per Minute (Raw WPM):**

```
Raw WPM = ((Total Occupied Positions) / 5) / (Time in Minutes)
```

- **Accuracy Percentage:**

```
Accuracy = ((Correct Characters) / (Total Occupied Positions)) * 100%
```

- **Consistency Score (RSD):**

```
RSD = StdDev(Inter-keystroke Intervals) / Mean(Intervals)
Consistency = Max(0, Min(100, 100 * (1 - RSD)))
```

- **Autotyper Anti-Cheat Detection:**

> Rejects submission if StdDev(Intervals) < 1.5ms (Mechanical Regularity)

# 8. Virtual Cat Companion Ecosystem

The TyProX Virtual Cat Companion engine (`CatCompanion.tsx`) provides 12 collectible animated cat companions, each with distinct color palettes, accessories, personality traits, and contextual reactive dialogues:

| Cat ID & Name | Personality & Color | Sample Dialogue |
|---|---|---|
| Silly Cheddar | Playful Orange (#FF9F43) | 100%? That deserves a whole virtual tuna slice! |
| Void Shadow | Mysterious Dark (#1E1E24) | Flawless. A glitch in the matrix, surely. |
| Kind Marshmallow | Sweet White (#F5F6FA) | You did absolutely perfect! I am so proud of you! |
| Coach Sergeant | Strict Grey (#7F8C8D) | Clean performance. Now hit the next test immediately! |
| Cyber Punk | Neon Cyan/Purple | Overclocked typing speed detected! Clean sync! |
| Golden Emperor | Legendary Gold | A performance worthy of the gold crown! |

# 9. Deployment, SEO & Performance Verification

TyProX is fully optimized for production deployment on Vercel or Node.js runtime environments:

- **SEO Optimization:** Dynamic metadata, semantic HTML5 elements, automated XML sitemap generator (`app/sitemap.ts`), and crawler rules (`app/robots.ts`).
- **Bundle Splitting:** Heavy charting libraries (Recharts) are lazy-loaded via Next.js `dynamic()` imports to minimize initial JavaScript bundle size.
- **Production Build Verification:** Evaluated via Next.js build compiler without runtime or TypeScript errors.

---

**End of Official Specification Document — TyProX Engineering Blueprint**